

```
In [ ]: # Import the packages
        # read the data visadata
        # cat num columns quickly
```

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

visa_df=pd.read_csv(r'C:\Users\omkar\OneDrive\Documents\Gen_AI\Data_files\Visada
```

```
In [5]: keys=visa_df.dtypes.keys()
values=visa_df.dtypes.values
```

```
In [7]: cat,num=[],[]
for i,j in zip(keys,values):
    if j=='object':
        cat.append(i)
    else:
        num.append(i)
```

```
In [9]: cat,num
```

```
Out[9]: (['case_id',
          'continent',
          'education_of_employee',
          'has_job_experience',
          'requires_job_training',
          'region_of_employment',
          'unit_of_wage',
          'full_time_position',
          'case_status'],
         ['no_of_employees', 'yr_of_estab', 'prevailing_wage'])
```

select_dtypes

```
In [19]: cat=visa_df.select_dtypes(include='object').columns
num=visa_df.select_dtypes(exclude='object').columns
visa_df.columns
```

```
Out[19]: Index(['case_id', 'continent', 'education_of_employee', 'has_job_experience',
               'requires_job_training', 'no_of_employees', 'yr_of_estab',
               'region_of_employment', 'prevailing_wage', 'unit_of_wage',
               'full_time_position', 'case_status'],
              dtype='object')
```

Categorical column analysis

```
In [21]: visa_df['continent'] # S
visa_df[['continent']] # DF
visa_df.continent # S
visa_df.iloc[:,1] # S
visa_df.iloc[:,[1]] # D
visa_df.loc[:, 'continent'] # S
visa_df.loc[:, ['continent']] # DF
```

Out[21]:

continent	
0	Asia
1	Asia
2	Asia
3	Asia
4	Africa
...	...
25475	Asia
25476	Asia
25477	Asia
25478	Asia
25479	Asia

25480 rows × 1 columns

```
In [27]: visa_df # DF
visa_df['continent'] # S
visa_df['continent'].values # raw data
```

Out[27]: array(['Asia', 'Asia', 'Asia', ..., 'Asia', 'Asia', 'Asia'], dtype=object)

continent

```
In [30]: visa_df['continent']
```

```
Out[30]: 0      Asia
1      Asia
2      Asia
3      Asia
4      Africa
...
25475  Asia
25476  Asia
25477  Asia
25478  Asia
25479  Asia
Name: continent, Length: 25480, dtype: object
```

```
In [32]: visa_df['continent'].unique()
```

Out[32]: array(['Asia', 'Africa', 'North America', 'Europe', 'South America',
 'Oceania'], dtype=object)

```
In [34]: visa_df['continent'].nunique()
```

Out[34]: 6

```
In [38]: visa_df['continent'].unique()
```

```
Out[38]: array(['Asia', 'Africa', 'North America', 'Europe', 'South America',
               'Oceania'], dtype=object)
```

frequency tables

```
In [59]: visa_df
con=visa_df['continent']=='Asia'
asia_count=np.sum(con)
asia_count
```

```
Out[59]: 16861
```

```
In [57]: len(visa_df[con])
```

```
Out[57]: 16861
```

```
In [63]: count=[]
con=visa_df['continent']=='Asia'
count.append(np.sum(con))
count
```

```
Out[63]: [16861]
```

```
In [ ]: count=[]

con=visa_df['continent']=='Asia'
count.append(np.sum(con))
count
```

```
In [67]: count=[]
for i in visa_df['continent'].unique():
    con=visa_df['continent']==i
    count.append(np.sum(con))
```

```
In [69]: count
```

```
Out[69]: [16861, 551, 3292, 3732, 852, 192]
```

```
In [71]: cont=visa_df['continent'].unique()
cont
```

```
Out[71]: array(['Asia', 'Africa', 'North America', 'Europe', 'South America',
               'Oceania'], dtype=object)
```

```
In [73]: count
```

```
Out[73]: [16861, 551, 3292, 3732, 852, 192]
```

```
In [77]: df1=pd.DataFrame(zip(cont,count),
                          columns=['Continent','No of Applicants'])
```

```
In [90]: df1.to_csv('continent_df.csv',index=False)
```

```
In [81]: df1
```

Out[81]:

	Continent	No of Applicants
0	Asia	16861
1	Africa	551
2	North America	3292
3	Europe	3732
4	South America	852
5	Oceania	192

	Continent	No of Applicants
0	Asia	16861
1	Africa	551
2	North America	3292
3	Europe	3732
4	South America	852
5	Oceania	192

In []:

value_counts

```
In [83]: visa_df
visa_df['continent']
visa_df['continent'].values()
visa_df['continent'].value_counts()
```

Out[83]:

continent	
Asia	16861
Europe	3732
North America	3292
South America	852
Africa	551
Oceania	192

Name: count, dtype: int64

```
In [87]: visa_df['continent'].value_counts(normalize=True)*100
```

Out[87]:

continent	
Asia	66.173469
Europe	14.646782
North America	12.919937
South America	3.343799
Africa	2.162480
Oceania	0.753532

Name: proportion, dtype: float64

```
In [ ]: # data
# we selected one column
# we represented in a tabualr format
# we need represent in a graphical
```

Bar chart

```
In [95]: keys=visa_df['continent'].value_counts().keys()
values=visa_df['continent'].value_counts().values
```

In [97]: keys

Out[97]:

```
Index(['Asia', 'Europe', 'North America', 'South America', 'Africa',
      'Oceania'],
      dtype='object', name='continent')
```

```
In [99]: values
```

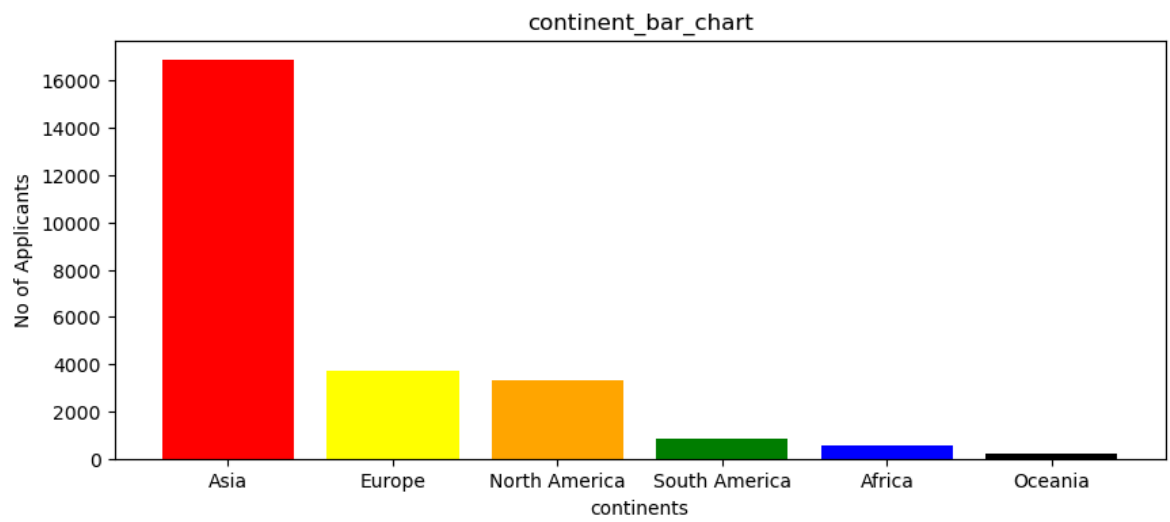
```
Out[99]: array([16861, 3732, 3292, 852, 551, 192], dtype=int64)
```

- frequency table we did from scratch also with method
 - **value_counts**
- we selected a cat and num data types
 - **select_dtypes**
- we can select a single column or multiple columns
 - **series or df**
- if any one give two list to you , you can able to create a dataframe
- clear difference between
 - dataframe
 - series
 - values : **numpy format**

```
In [4]: keys=visa_df['continent'].value_counts().keys()
values=visa_df['continent'].value_counts().values
```

```
In [38]: plt.figure(figsize=(10,4))
colors=['red','yellow','orange','green','blue','black']
plt.bar(keys,values,color=colors)
plt.xlabel('continents')
plt.ylabel('No of Applicants')
plt.title('continent_bar_chart')
plt.savefig('continent.jpg')
plt.show()

#           txt           csv           image
# where    diff or same  diff or same  diff or same
# what     data          data          image
# extention .txt         data.csv     image.png or image.jpg
```



```
In [22]: import matplotlib
dir(matplotlib)
```

```
Out[22]: ['ExecutableNotFoundError',
'MatplotlibDeprecationWarning',
'MutableMapping',
'Parameter',
'Path',
'RcParams',
'_ExecInfo',
'_VersionInfo',
'__all__',
'__bibtex__',
'__builtins__',
'__cached__',
'__doc__',
'__file__',
'__getattr__',
'__loader__',
'__name__',
'__package__',
'__path__',
'__spec__',
'_add_data_doc',
'_afm',
'_api',
'_blocking_input',
'_c_internal_utils',
'_check_versions',
'_cm',
'_cm_listed',
'_color_data',
'_constrained_layout',
'_deprecated_ignore_map',
'_deprecated_map',
'_deprecated_remain_as_none',
'_docstring',
'_ensure_handler',
'_enums',
'_fontconfig_pattern',
'_get_config_or_cache_dir',
'_get_executable_info',
'_get_ssl_context',
'_get_version',
'_get_xdg_cache_dir',
'_get_xdg_config_dir',
'_image',
'_init_tests',
'_label_from_arg',
'_layoutgrid',
'_log',
'_logged_cached',
'_mathtext',
'_mathtext_data',
'_open_file_or_url',
'_parse_to_version_info',
'_path',
'_preprocess_data',
'_pylab_helpers',
'_rc_params_in_file',
'_replacer',
'_text_helpers',
'_tight_bbox',
```

'_tight_layout',
'_val_or_rc',
'_version',
'artist',
'atexit',
'axes',
'axis',
'backend_bases',
'backend_managers',
'backend_tools',
'backends',
'bezier',
'category',
'cbook',
'cm',
'collections',
'color_sequences',
'colorbar',
'colormaps',
'colors',
'container',
'contextlib',
'contour',
'cycler',
'dates',
'defaultParams',
'dviread',
'figure',
'font_manager',
'ft2font',
'functools',
'get_backend',
'get_cachedir',
'get_configdir',
'get_data_path',
'gridspec',
'hatch',
'image',
'importlib',
'inspect',
'interactive',
'is_interactive',
'layout_engine',
'legend',
'legend_handler',
'lines',
'locale',
'logging',
'markers',
'mathtext',
'matplotlib_fname',
'mlab',
'namedtuple',
'offsetbox',
'os',
'parse_version',
'patches',
'path',
'pprint',
'projections',

```
'pyplot',  
'quiver',  
'rc',  
'rcParams',  
'rcParamsDefault',  
'rcParamsOrig',  
'rc_context',  
'rc_file',  
'rc_file_defaults',  
'rc_params',  
'rc_params_from_file',  
'rcdefaults',  
'rcsetup',  
're',  
'sanitize_sequence',  
'scale',  
'set_loglevel',  
'shutil',  
'spines',  
'stackplot',  
'streamplot',  
'style',  
'subprocess',  
'sys',  
'table',  
'tempfile',  
'texmanager',  
'text',  
'textpath',  
'ticker',  
'transforms',  
'tri',  
'units',  
'use',  
'validate_backend',  
'widgets']
```

```
In [26]: from matplotlib import colors
```

```
In [28]: colors.CSS4_COLORS
```

```
Out[28]: {'aliceblue': '#F0F8FF',
          'antiquewhite': '#FAEBD7',
          'aqua': '#00FFFF',
          'aquamarine': '#7FFFD4',
          'azure': '#F0FFFF',
          'beige': '#F5F5DC',
          'bisque': '#FFE4C4',
          'black': '#000000',
          'blanchedalmond': '#FFEBCD',
          'blue': '#0000FF',
          'blueviolet': '#8A2BE2',
          'brown': '#A52A2A',
          'burlywood': '#DEB887',
          'cadetblue': '#5F9EA0',
          'chartreuse': '#7FFF00',
          'chocolate': '#D2691E',
          'coral': '#FF7F50',
          'cornflowerblue': '#6495ED',
          'cornsilk': '#FFF8DC',
          'crimson': '#DC143C',
          'cyan': '#00FFFF',
          'darkblue': '#00008B',
          'darkcyan': '#008B8B',
          'darkgoldenrod': '#B8860B',
          'darkgray': '#A9A9A9',
          'darkgreen': '#006400',
          'darkgrey': '#A9A9A9',
          'darkkhaki': '#BDB76B',
          'darkmagenta': '#8B008B',
          'darkolivegreen': '#556B2F',
          'darkorange': '#FF8C00',
          'darkorchid': '#9932CC',
          'darkred': '#8B0000',
          'darksalmon': '#E9967A',
          'darkseagreen': '#8FBC8F',
          'darkslateblue': '#483D8B',
          'darkslategray': '#2F4F4F',
          'darkslategrey': '#2F4F4F',
          'darkturquoise': '#00CED1',
          'darkviolet': '#9400D3',
          'deeppink': '#FF1493',
          'deepskyblue': '#00BFFF',
          'dimgray': '#696969',
          'dimgrey': '#696969',
          'dodgerblue': '#1E90FF',
          'firebrick': '#B22222',
          'floralwhite': '#FFFAF0',
          'forestgreen': '#228B22',
          'fuchsia': '#FF00FF',
          'gainsboro': '#DCDCDC',
          'ghostwhite': '#F8F8FF',
          'gold': '#FFD700',
          'goldenrod': '#DAA520',
          'gray': '#808080',
          'green': '#008000',
          'greenyellow': '#ADFF2F',
          'grey': '#808080',
          'honeydew': '#F0FFF0',
          'hotpink': '#FF69B4',
          'indianred': '#CD5C5C',
```

'indigo': '#4B0082',
'ivory': '#FFFFF0',
'khaki': '#F0E68C',
'lavender': '#E6E6FA',
'lavenderblush': '#FFF0F5',
'lawngreen': '#7CFC00',
'lemonchiffon': '#FFFFACD',
'lightblue': '#ADD8E6',
'lightcoral': '#F08080',
'lightcyan': '#E0FFFF',
'lightgoldenrodyellow': '#FAFAD2',
'lightgray': '#D3D3D3',
'lightgreen': '#90EE90',
'lightgrey': '#D3D3D3',
'lightpink': '#FFB6C1',
'lightsalmon': '#FFA07A',
'lightseagreen': '#20B2AA',
'lightskyblue': '#87CEFA',
'lightslategray': '#778899',
'lightslategrey': '#778899',
'lightsteelblue': '#B0C4DE',
'lightyellow': '#FFFFE0',
'lime': '#00FF00',
'limegreen': '#32CD32',
'linen': '#FAF0E6',
'magenta': '#FF00FF',
'maroon': '#800000',
'mediumaquamarine': '#66CDAA',
'mediumblue': '#0000CD',
'mediumorchid': '#BA55D3',
'mediumpurple': '#9370DB',
'mediumseagreen': '#3CB371',
'mediumslateblue': '#7B68EE',
'mediumspringgreen': '#00FA9A',
'mediumturquoise': '#48D1CC',
'mediumvioletred': '#C71585',
'midnightblue': '#191970',
'mintcream': '#F5FFFA',
'mistyrose': '#FFE4E1',
'moccasin': '#FFE4B5',
'navajowhite': '#FFDEAD',
'navy': '#000080',
'oldlace': '#FDF5E6',
'olive': '#808000',
'olivedrab': '#6B8E23',
'orange': '#FFA500',
'orangered': '#FF4500',
'orchid': '#DA70D6',
'palegoldenrod': '#EEE8AA',
'palegreen': '#98FB98',
'paleturquoise': '#AFEEEE',
'palevioletred': '#DB7093',
'papayawhip': '#FFEFD5',
'peachpuff': '#FFDAB9',
'peru': '#CD853F',
'pink': '#FFC0CB',
'plum': '#DDA0DD',
'powderblue': '#B0E0E6',
'purple': '#800080',
'rebeccapurple': '#663399',

```

'red': '#FF0000',
'rosybrown': '#BC8F8F',
'royalblue': '#4169E1',
'saddlebrown': '#8B4513',
'salmon': '#FA8072',
'sandybrown': '#F4A460',
'seagreen': '#2E8B57',
'seashell': '#FFF5EE',
'sienna': '#A0522D',
'silver': '#C0C0C0',
'skyblue': '#87CEEB',
'slateblue': '#6A5ACD',
'slategray': '#708090',
'slategrey': '#708090',
'snow': '#FFFAFA',
'springgreen': '#00FF7F',
'steelblue': '#4682B4',
'tan': '#D2B48C',
'teal': '#008080',
'thistle': '#D8BFD8',
'tomato': '#FF6347',
'turquoise': '#40E0D0',
'violet': '#EE82EE',
'wheat': '#F5DEB3',
'white': '#FFFFFF',
'whitesmoke': '#F5F5F5',
'yellow': '#FFFF00',
'yellowgreen': '#9ACD32'}

```

```
In [42]: cat= visa_df.select_dtypes(include='object').columns
```

```
In [44]: cat
```

```
Out[44]: Index(['case_id', 'continent', 'education_of_employee', 'has_job_experience',
               'requires_job_training', 'region_of_employment', 'unit_of_wage',
               'full_time_position', 'case_status'],
              dtype='object')
```

```
In [46]: cat[1:]
```

```
Out[46]: Index(['continent', 'education_of_employee', 'has_job_experience',
               'requires_job_training', 'region_of_employment', 'unit_of_wage',
               'full_time_position', 'case_status'],
              dtype='object')
```

```
In [48]: visa_df['case_id'].nunique()
# we should avoid
```

```
Out[48]: 25480
```

```
In [52]: len(cat[1:])
```

```
Out[52]: 8
```

```
In [50]: keys=visa_df['continent'].value_counts().keys()
values=visa_df['continent'].value_counts().values
plt.figure(figsize=(10,4))
plt.bar(keys,values)
plt.xlabel('Label')
```

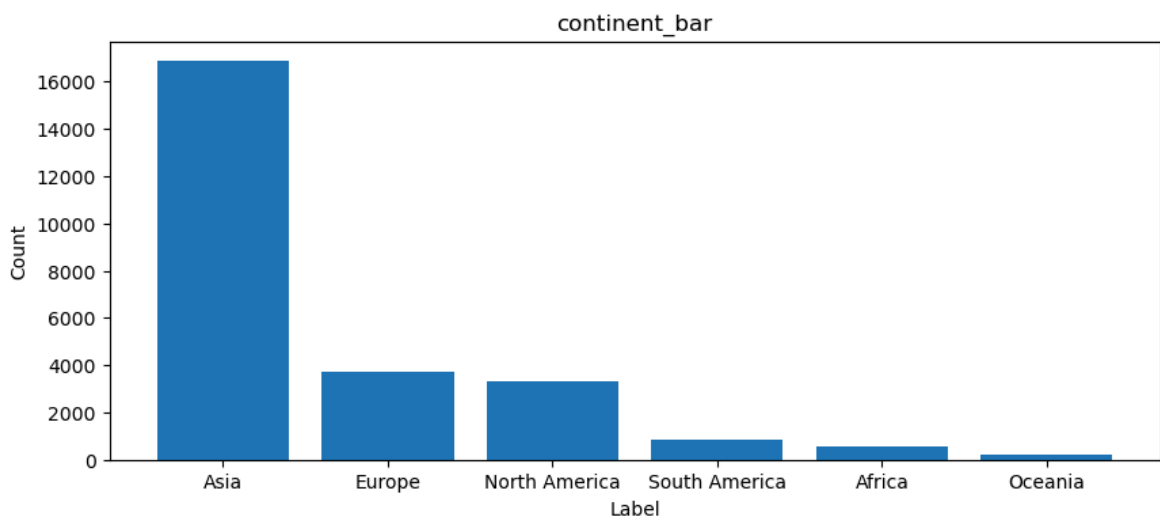
```

plt.ylabel('Count')
plt.title('continent_bar')
plt.savefig('continent_bar.jpg')
#####
keys=visa_df['education_of_employee'].value_counts().keys()
values=visa_df['education_of_employee'].value_counts().values
plt.figure(figsize=(10,4))
plt.bar(keys,values)
plt.xlabel('Label')
plt.ylabel('Count')
plt.title('education_of_employee_bar')
plt.savefig('education_of_employee_bar.jpg')

keys=visa_df[i].value_counts().keys()
values=visa_df[i].value_counts().values
plt.figure(figsize=(10,4))
plt.bar(keys,values)
plt.xlabel('Label')
plt.ylabel('Count')
plt.title(f'{i}_bar')
plt.savefig(f'{i}_bar.jpg')

# who is i?= categorical columns
#

```



```

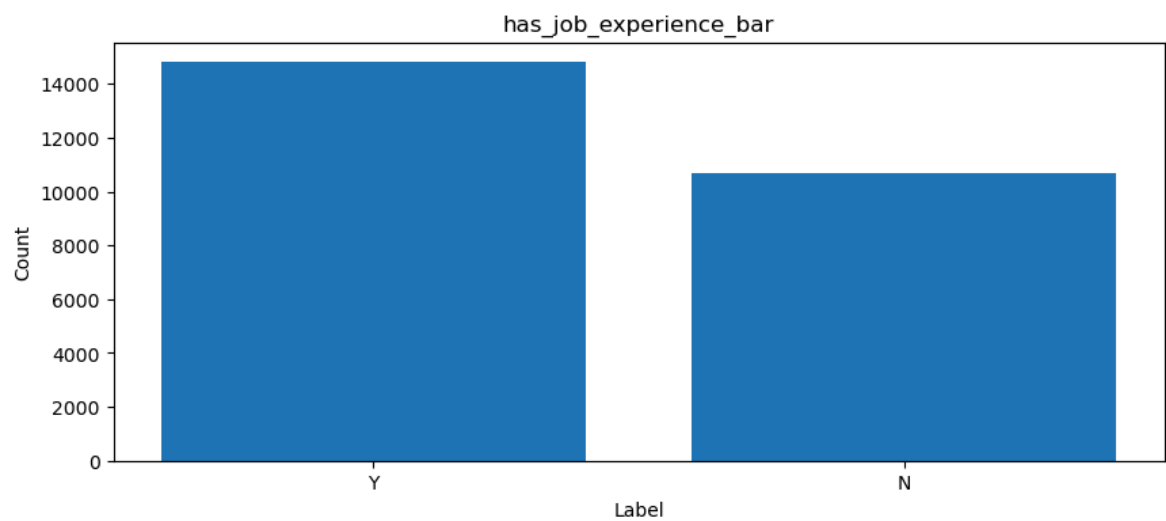
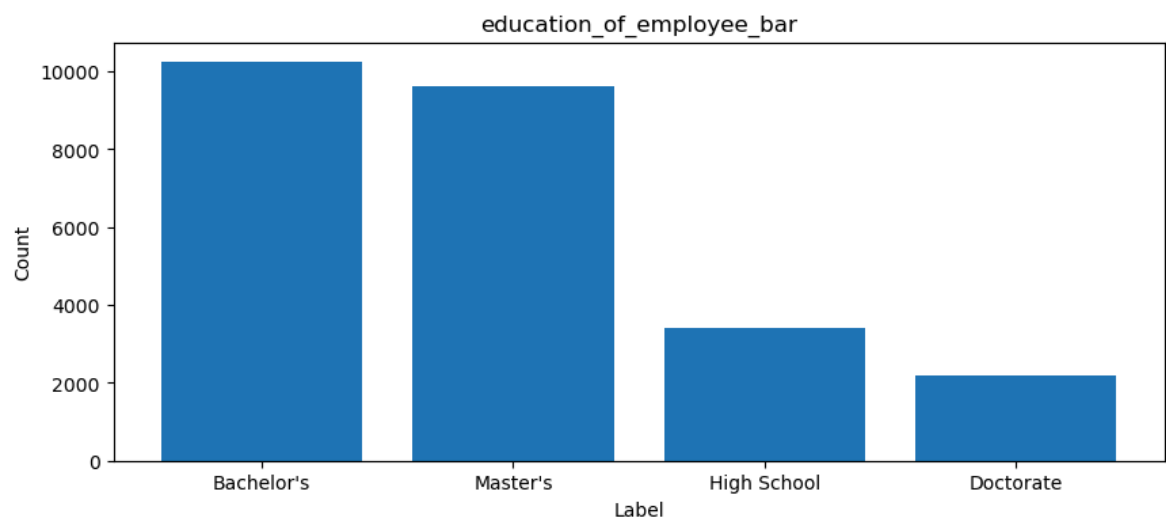
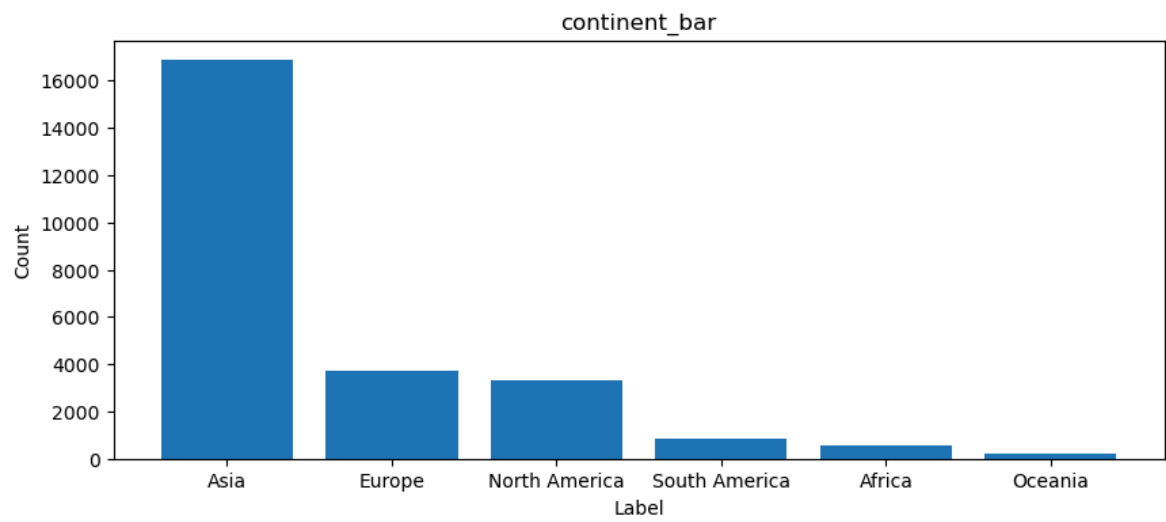
In [ ]: # import
        # read
        # cat num

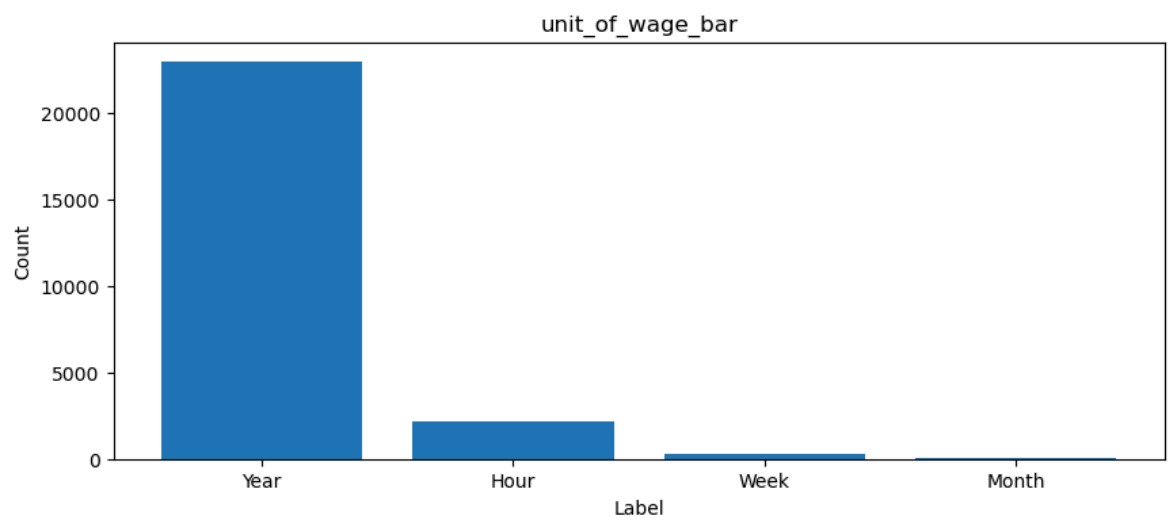
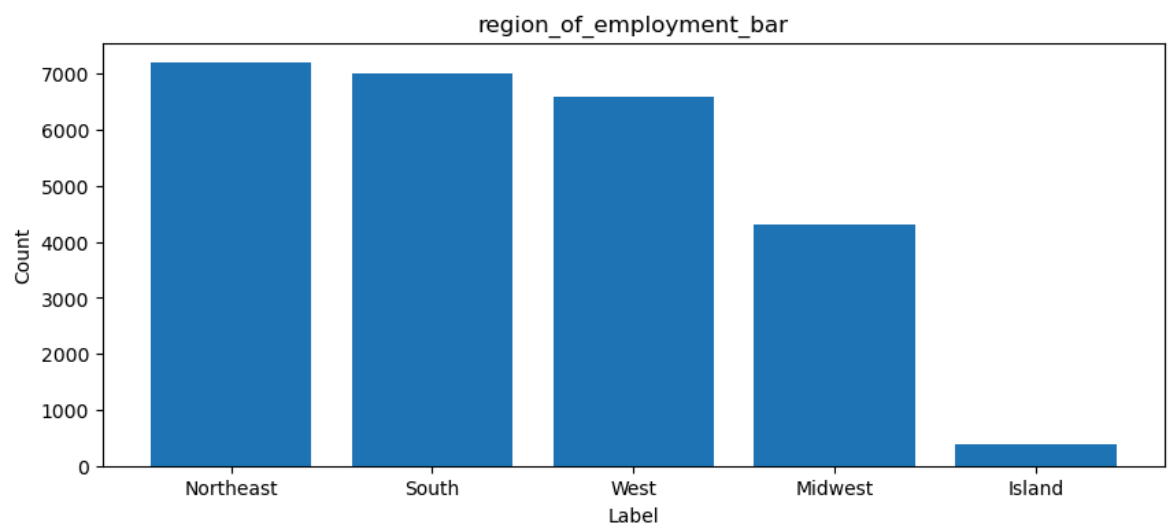
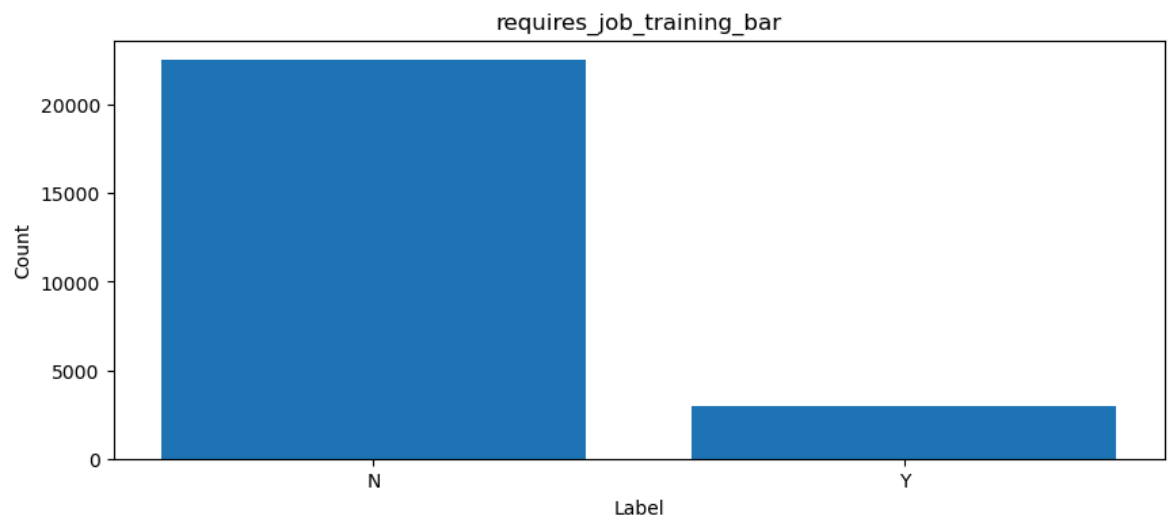
```

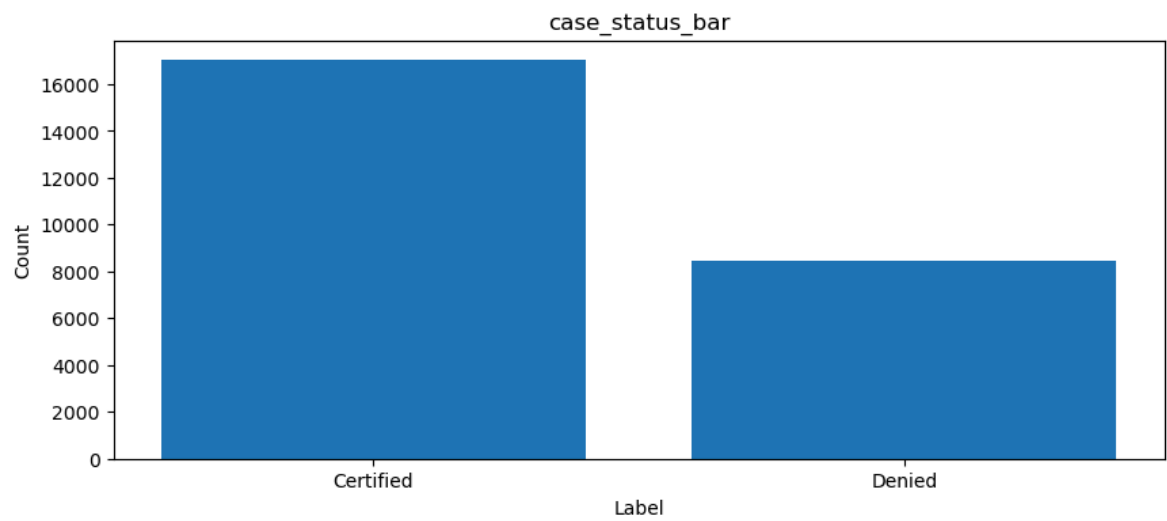
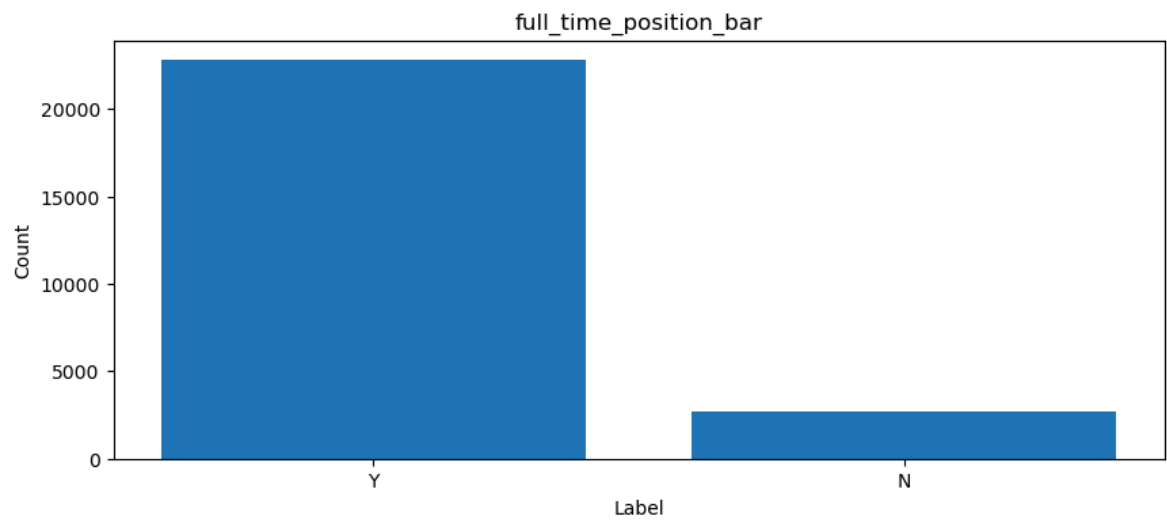
```

In [54]: for i in cat[1:]:
        keys=visa_df[i].value_counts().keys()
        values=visa_df[i].value_counts().values
        plt.figure(figsize=(10,4))
        plt.bar(keys,values)
        plt.xlabel('Label')
        plt.ylabel('Count')
        plt.title(f'{i}_bar')
        plt.savefig(f'{i}_bar.jpg')

```

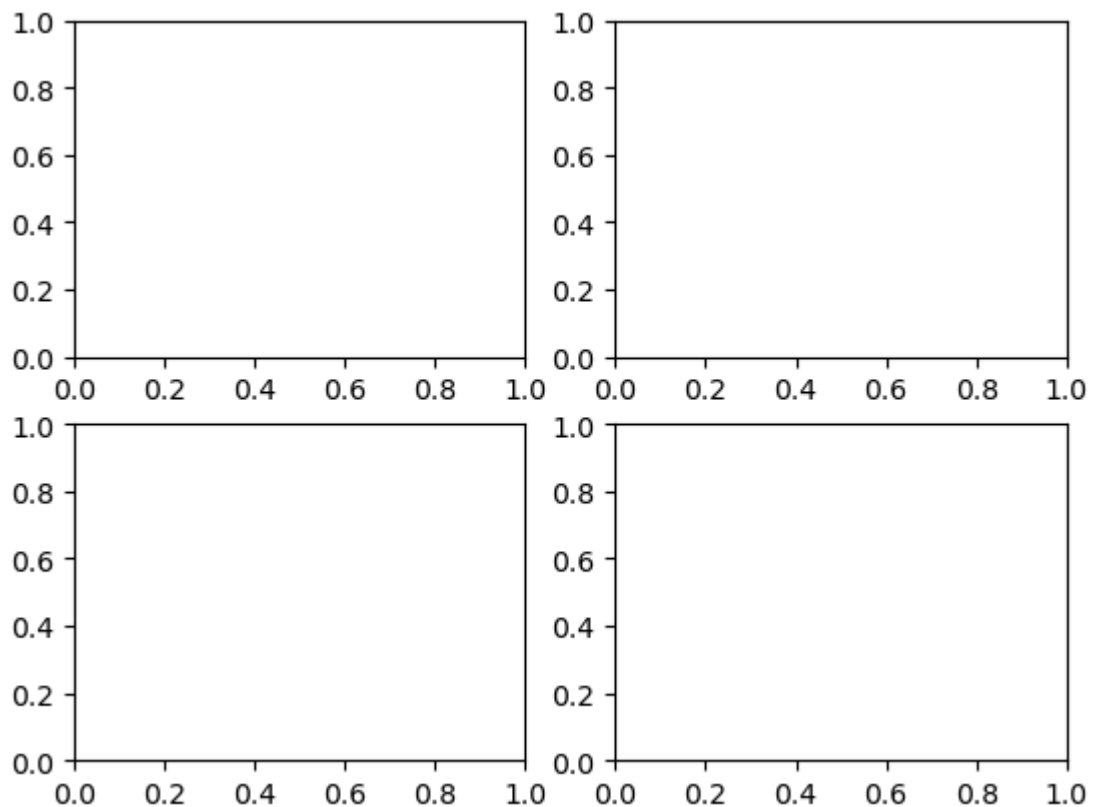






```
In [56]: plt.subplot(2,2,1)
plt.subplot(2,2,2)
plt.subplot(2,2,3)
plt.subplot(2,2,4)
```

Out[56]: <Axes: >



In [58]: `plt.subplot(2,2,5)`

```
-----
ValueError                                Traceback (most recent call last)
Cell In[58], line 1
----> 1 plt.subplot(2,2,5)

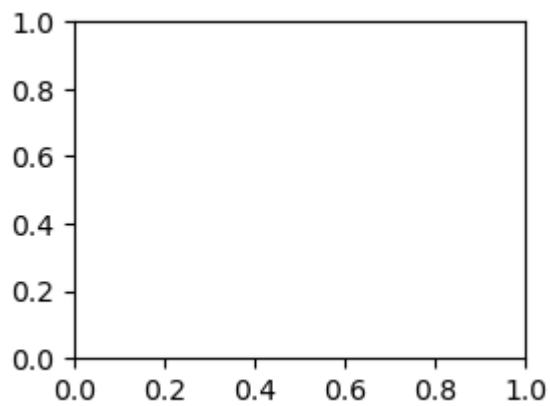
File E:\Anaconda\Lib\site-packages\matplotlib\pyplot.py:1534, in subplot(*args, *
kwargs)
    1531 fig = gcf()
    1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
    1536 for ax in fig.axes:
    1537     # If we found an Axes at the position, we can reuse it if the user pa
ssed no
    1538     # kwargs or if the Axes class and kwargs are identical.
    1539     if (ax.get_subplotspec() == key
    1540         and (kwargs == {}
    1541             or (ax._projection_init
    1542                 == fig._process_projection_requirements(**kwargs))))):

File E:\Anaconda\Lib\site-packages\matplotlib\gridspec.py:589, in SubplotSpec._fr
om_subplot_args.figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 4, not 5
<Figure size 640x480 with 0 Axes>
```

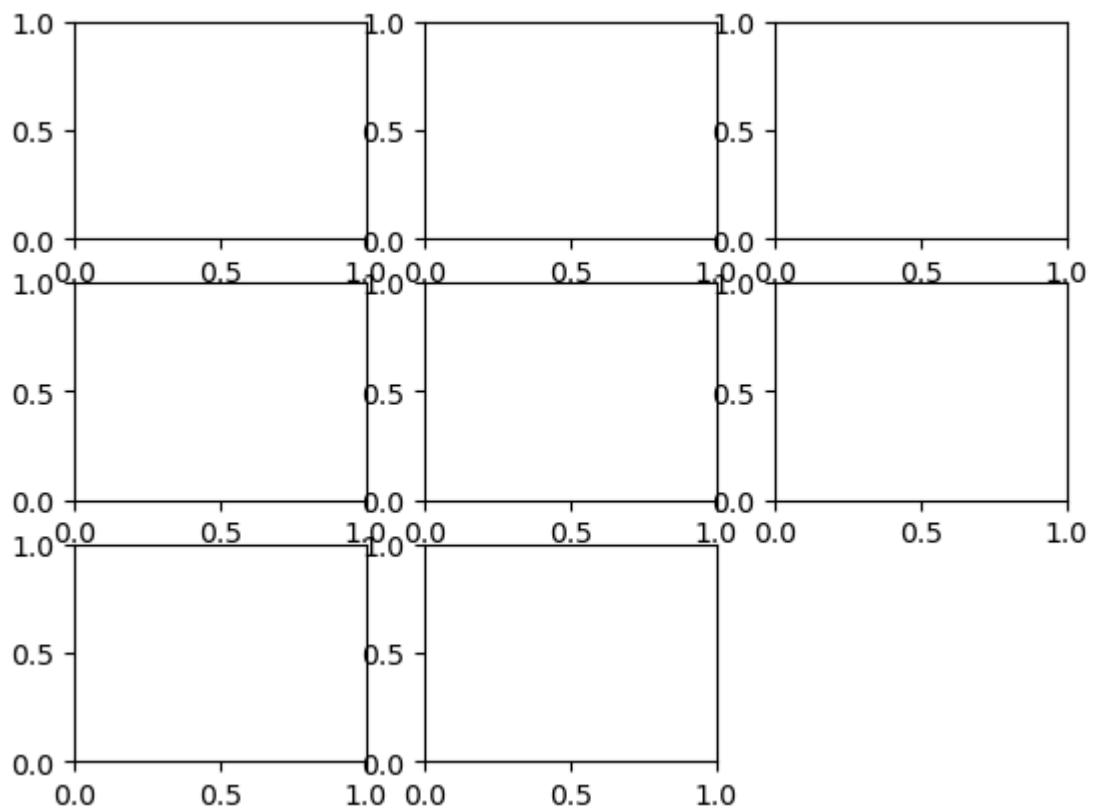
```
In [60]: plt.subplot(2,2,3)
```

Out[60]: <Axes: >



```
In [ ]: # we need to plot 8  
# 1x8 not rec  
# 8x1 (prev) not rec  
# 2x4 okay  
# 4x2 not  
# 3x3 okay
```

```
In [62]: plt.subplot(3,3,1)  
plt.subplot(3,3,2)  
  
for i in range(len(cat[1:])):  
    plt.subplot(3,3,i+1)
```



```
In [64]: for i,j in enumerate(cat[1:]):  
    print(i,j)
```

```

0 continent
1 education_of_employee
2 has_job_experience
3 requires_job_training
4 region_of_employment
5 unit_of_wage
6 full_time_position
7 case_status

```

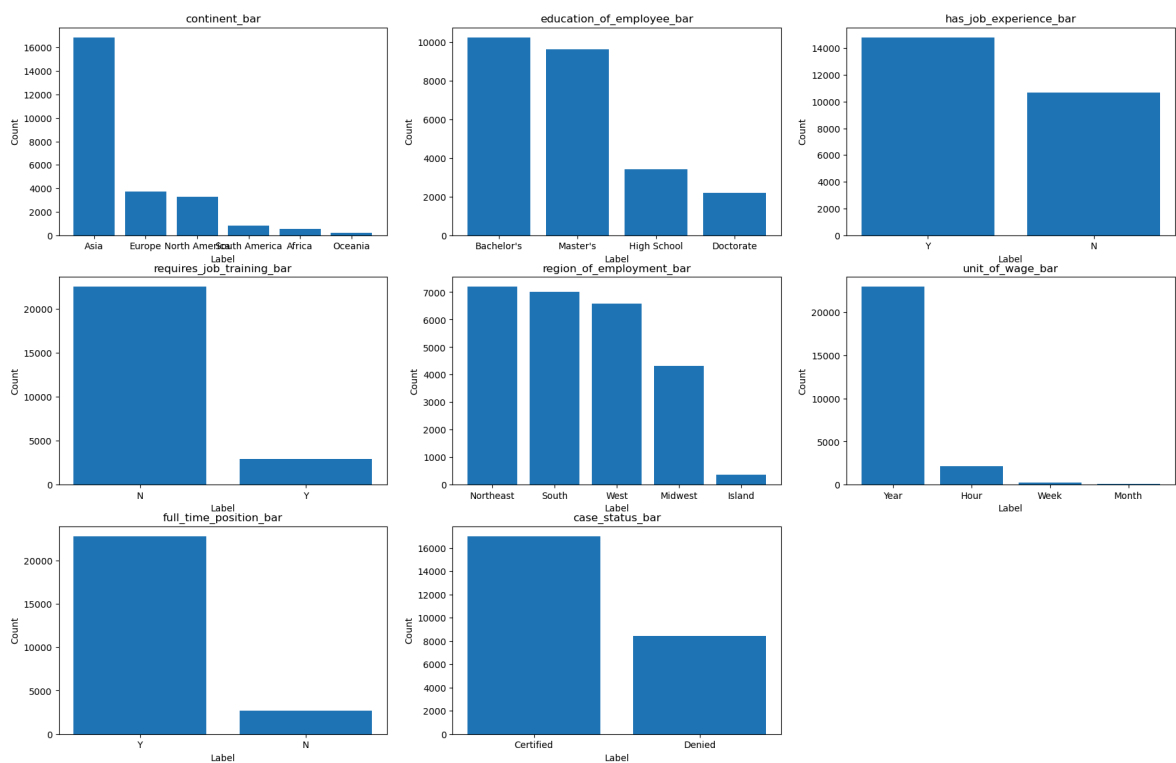
```

In [80]: plt.figure(figsize=(22,14))

for i,j in enumerate(cat[1:]): # i = 0
    keys=visa_df[j].value_counts().keys()
    values=visa_df[j].value_counts().values
    #####
    plt.subplot(3,3,i+1)
    plt.bar(keys,values)
    #####
    plt.xlabel('Label')
    plt.ylabel('Count')
    plt.title(f'{j}_bar')
    plt.savefig(f'{j}_bar.jpg')

plt.show()

```



In []: